Service-oriented Software Engineering (SoSE)

(must have) Advanced Software Engineering ingredients towards Service Orientation

(2/3 and 3/3)

Marco Autili

University of L'Aquila

marco.autili@univaq.it

http://people.disim.univaq.it/marco.autili/

About these slides

- These slides are mostly based on the original slides (publicly available from the official sites of the two books that I will adopt as reference textbooks for this course)

- The complete responsibility of the changes made by myself is addressable to me

Topics covered by these slides

Software reuse

- Reuse-based Software Engineering
- The reuse landscape
- Implementing software reuse

Component-based software engineering

- Components and component models
- CBSE processes
- Component composition

Distributed software engineering

- Distributed systems issues
- Client–server computing
- Architectural patterns for distributed systems
- Software as a service

Component-based development

- Component-based software engineering (CBSE) is an approach to software development that relies on the reuse of entities called 'software components'.

- It emerged from the failure of object-oriented development to support effective reuse. Single object classes are too detailed and specific.

- Components are more abstract than object classes and can exist as stand-alone entities.

Component standards

- Standards need to be established so that components can communicate with each other and inter-operate.

- Unfortunately, in the past, several competing and different proprietary component standards were established:
  - Sun's Enterprise Java Beans
  - Microsoft's COM/DCOM and .NET
  - CORBA
  - … and related components models (see later slides)

In practice, these multiple standards have hindered the uptake of CBSE. It is impossible for components developed using different approaches to seamlessly work together

Component definition(s)

Councill and Heinmann

- A software component is a software element that conforms to a component model and can be independently deployed and composed without modification according to a composition standard.

Szyperski

- A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third-parties.

CBSE vs SoSE

As we will see later on the course

- from the outset, services have been based around **standards** so there are no communication problems between services offered by different vendors.

- system **performance may be slower with services** but this approach is replacing CBSE in many systems.

Component characteristics

Component characteristic	Description
Composable	For a component to be composable, all external interactions must take place through publicly defined interfaces. In addition, it must provide external access to information about itself, such as its methods and attributes.
Deployable	To be deployable, a component has to be self-contained. It must be able to operate as a stand-alone entity on a component platform that provides an implementation of the component model. This usually means that the component is binary and does not have to be compiled before it is deployed. If a component is implemented as a service, it does not have to be deployed by a user of a component. Rather, it is deployed by the service provider.

Component characteristics

Component characteristic	Description
Documented	Components have to be fully documented so that potential users can decide whether or not the components meet their needs. The syntax and, ideally, the semantics of all component interfaces should be specified.
Independent	A component should be independent. It should be possible to compose and deploy it without having to use other specific components. In situations where the component needs externally provided services, these should be explicitly set out in a 'requires' interface specification.
Standardized	Component standardization means that a component used in a CBSE process has to conform to a standard component model. This model may define component interfaces, component metadata, documentation, composition, and deployment.

Component as a service provider

- The component is an independent, executable entity. It does not have to be compiled with other components before it is used.

- The "services" offered by a component are made available through an interface and all component interactions take place through that interface.

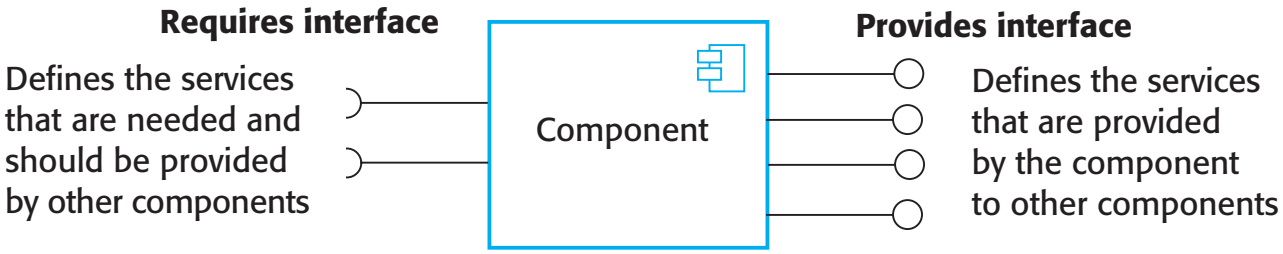
- The component interface is expressed in terms of parameterized operations and its internal state is (should be) never exposed.

Pay attention, wrt service-orientation, the word "service" is misleading here!

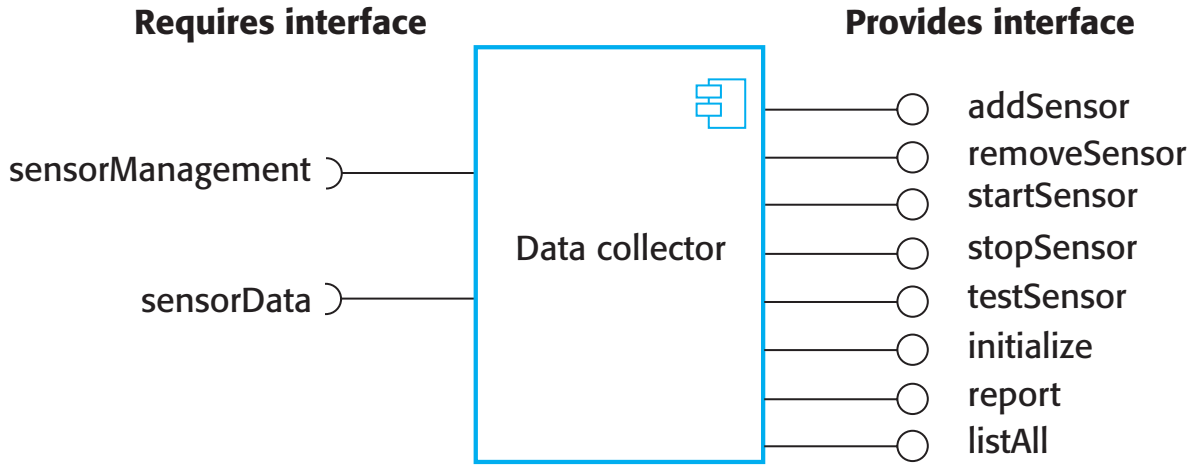
Component interfaces

- Provide interface
  - Defines the "services" that are provided by the component to other components.
  - This interface, essentially, is the component API. It defines the methods that can be called by a user of the component.

- Require interface
  - Defines the "services" that are required by the component so to be able to execute as specified.
  - This does not compromise the independence or deployability of a component because the 'requires' interface does not define how these "services" should be provided.



Note UML notation. Ball and sockets can fit together.



A model of a data collector component

Component access

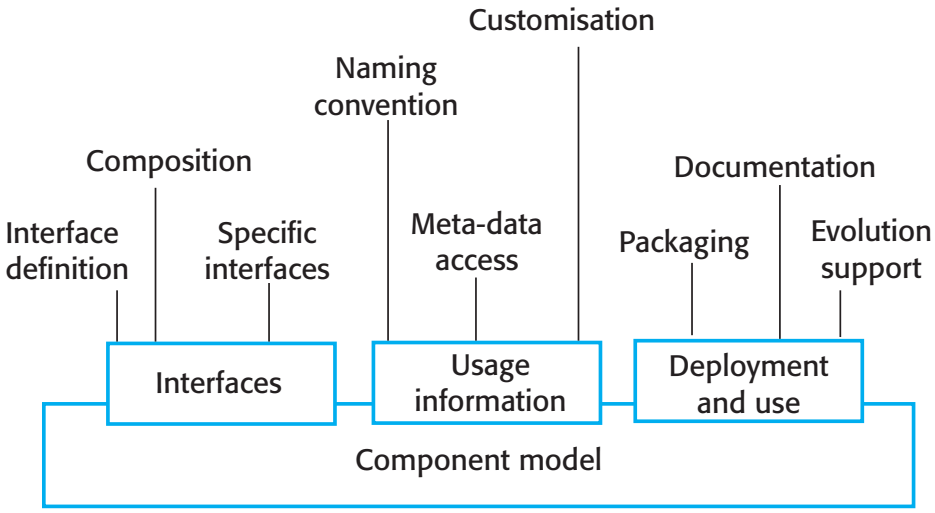
Components are accessed using remote procedure calls (RPCs).

Each component has a unique identifier (usually a URL) and can be referenced from any networked computer.

Therefore, it can be called in a similar way as a procedure or method running on a local computer.

Component models

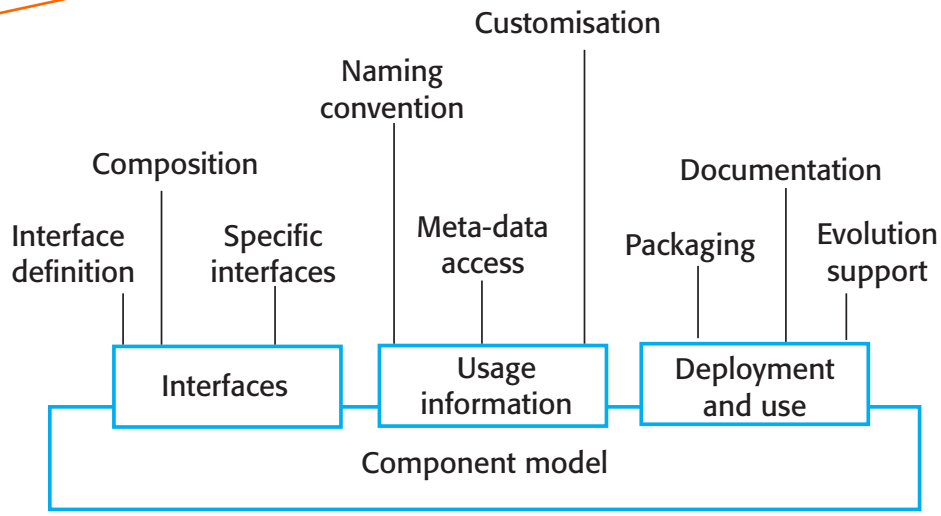
- A component model is a definition of standards for component implementation, interface specification, documentation and deployment. Examples of component models:
  - EJB model (Enterprise Java Beans)
  - COM+ model (.NET model)
  - Corba Component Model



- Importantly, the component model specifies how interfaces should be defined and the elements that should be included in an interface definition.

Component models

Unfortunately, proprietary (competing and different) component standards and component models introduce technological barriers to interoperability!



Elements of a component model

- Interfaces
  - Components are defined by specifying their interfaces. The component model specifies how the interfaces should be defined and the elements, such as operation names, parameters and exceptions, which should be included in the interface definition.
- Usage (access)
  - In order for components to be distributed and accessed remotely, they need to have a unique name or handle associated with them. This has to be globally unique.
- Deployment
  - The component model includes a specification of how components should be packaged for deployment as independent, executable entities.

Unfortunately, proprietary (competing and different) component standards and component models introduce technological barriers to interoperability!

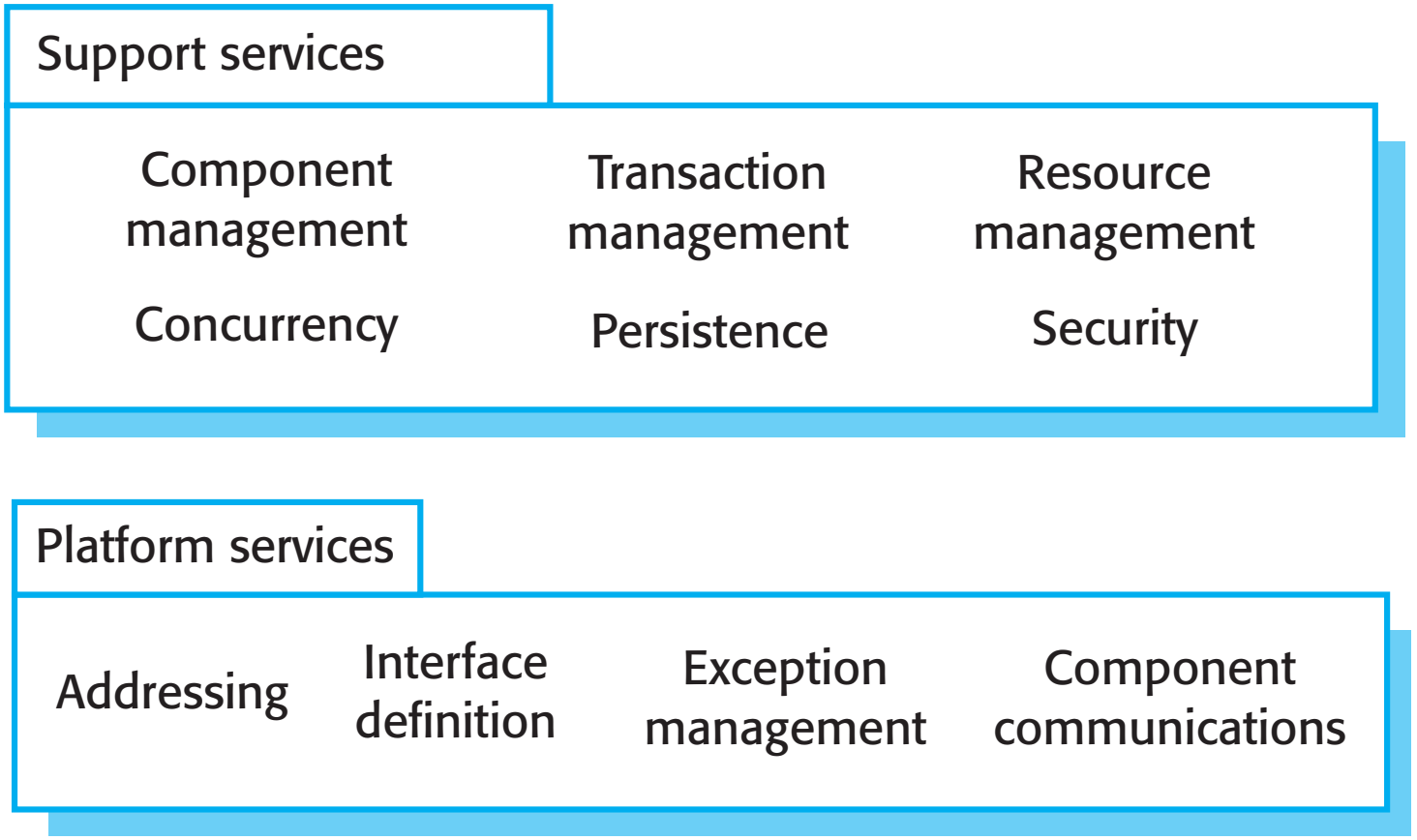
Middleware support

- Component models are the basis for middleware that provides support for executing components.

- Component model implementations provide:
  - Platform services that allow components written according to the model to communicate;
  - Support services that are application-independent services used by different components.

- To use services provided by a model, components are deployed in a container. This is a set of interfaces used to access the service implementations.

Middleware services defined in a component model



CBSE processes

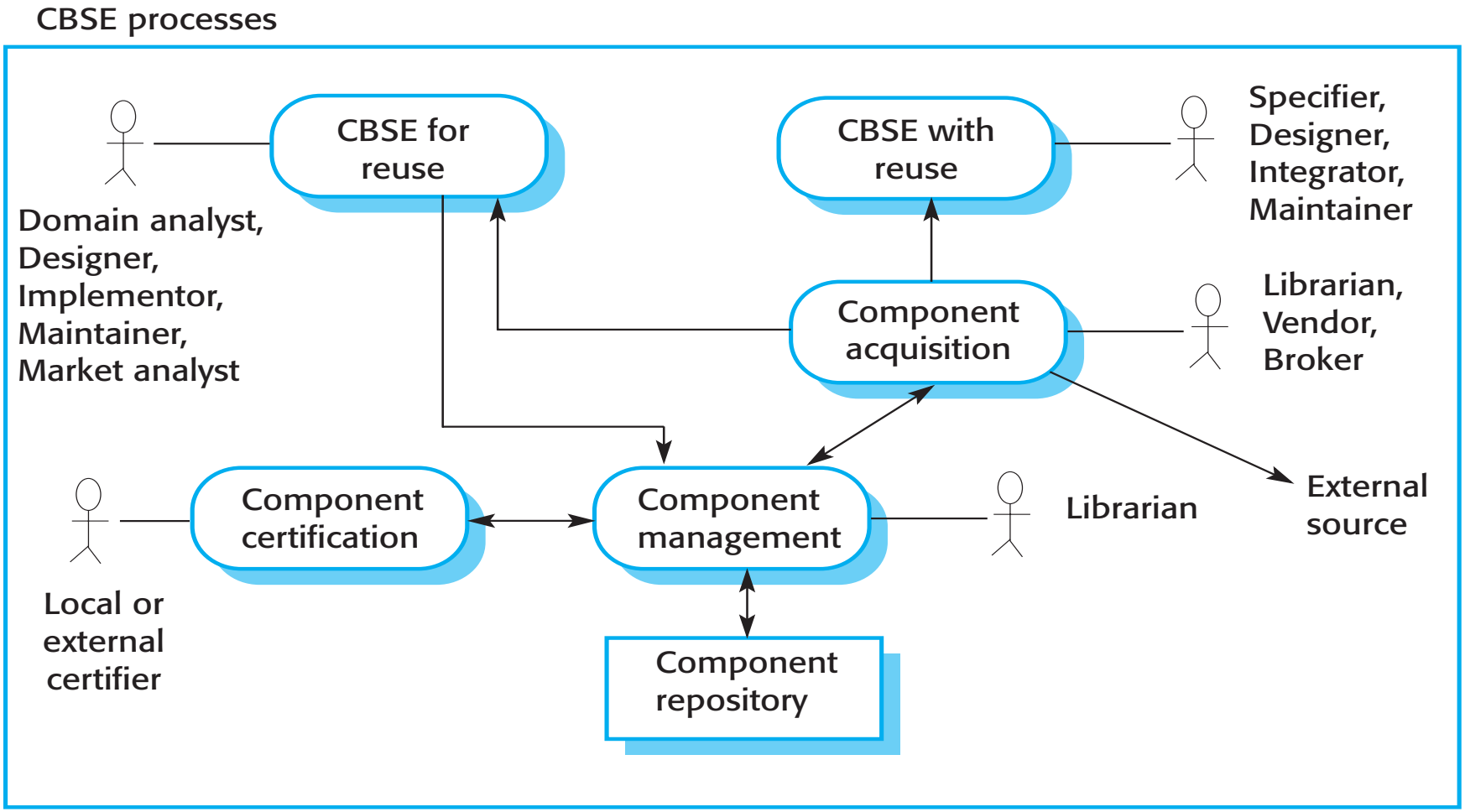
- CBSE processes are software processes that support component-based software engineering.
  - They take into account the possibilities of reuse and the different process activities involved in developing and using reusable components.

Similarly to SoSE

- Development for reuse
  - This process is concerned with developing components (or services) that are amenable to be reused in other applications.
  - It usually involves generalizing existing components.

- Development with reuse
  - This process is the process of developing new applications using existing components (and services).

CBSE processes



Legacy system components

Similarly to SoSE

- Existing legacy systems that fulfil a useful business function can be re-packaged as components for reuse.

  In principle, re-packing (or component-based wrapping) share similarity with wrapping legacy software through Web Services (WS-based wrapping)… in substance, it is different

- This involves writing a wrapper component that implements provides and requires interfaces then accesses the legacy system.

- Although costly, this can be much less expensive than rewriting the legacy system.

Ariane launcher failure – validation failure?

- In 1996, the 1st test flight of the Ariane 5 rocket ended in disaster when the launcher went out of control 37 seconds after take off.

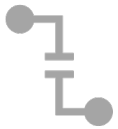
- The problem was due to a reused component from a previous version of the launcher (the Inertial Navigation System) that failed because assumptions made when that component was developed did not hold for Ariane 5.

- The functionality that failed in this component was not required in Ariane 5.

Component composition

- The process of assembling components to create a system.
- Composition involves integrating components with each other and with the component infrastructure.
- Normally you have to write 'glue code' to integrate components.

Types of composition (next slide)



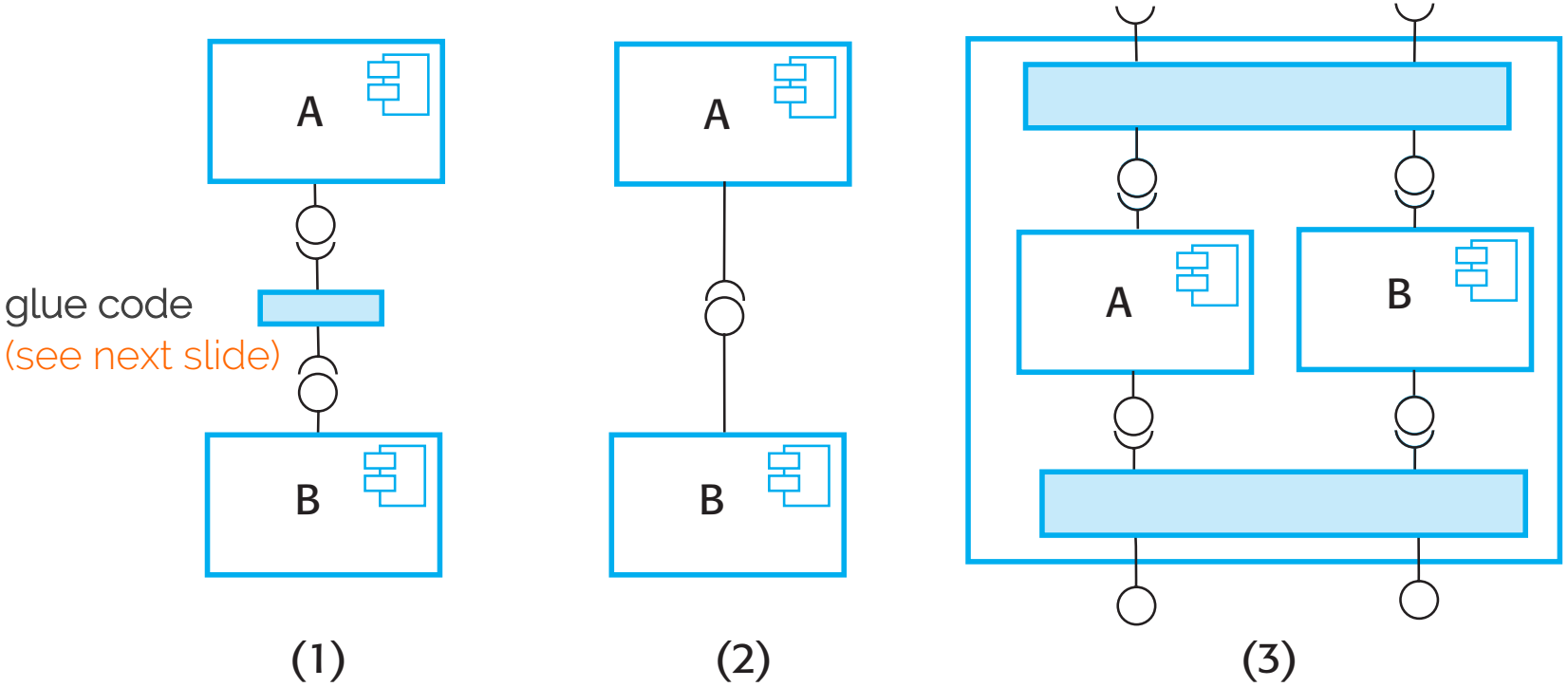
Sequential composition (1) where the composed components are executed in sequence. This involves composing the provides interfaces of each component.

Hierarchical composition (2) where one component calls on the services of another. The provides interface of one component is composed with the requires interface of another.

Additive composition (3) where the interfaces of two components are put together to create a new component. Provides and requires interfaces of integrated component is a combination of interfaces of constituent components.

Types of component composition

- Again, similarly to service orientation…



Glue code

- Code that allows components to work together

- If A and B are composed sequentially, then glue code has to call A, collect its results then call B using these results, transforming them into the format required by B.

- Glue code may be used to resolve interface and behavioral incompatibilities, to enforce coordination, etc.

Interface incompatibility

Parameter incompatibility

operations have the same name but are of different types.

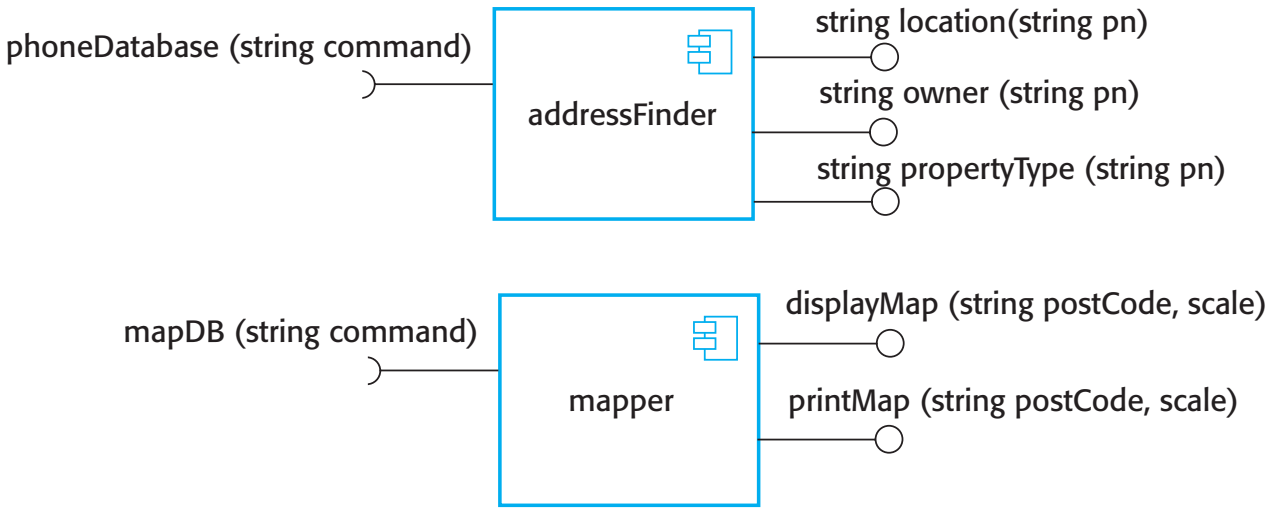
Operation incompatibility

the names of operations in the composed interfaces are different.

Operation incompleteness

provides interface of one component is a subset of the requires interface of another.

Components with incompatible interfaces



- Adaptor - Address the problem of component incompatibility by reconciling the interfaces of the components that are composed.

- Different types of adaptor are required depending on the type of composition.

- An addressFinder and a mapper component may be composed through an adaptor that strips the postal code from an address and passes this to the mapper component (see next slide...).

Composition through an adaptor

- The component postCodeStripper is the adaptor that facilitates the sequential composition of addressFinder and mapper components.

```
address = addressFinder.location (phonenumber) ;
postCode = postCodeStripper.getPostCode (address) ;
mapper.displayMap(postCode, 10000)
```

An adaptor linking a data collector and a sensor

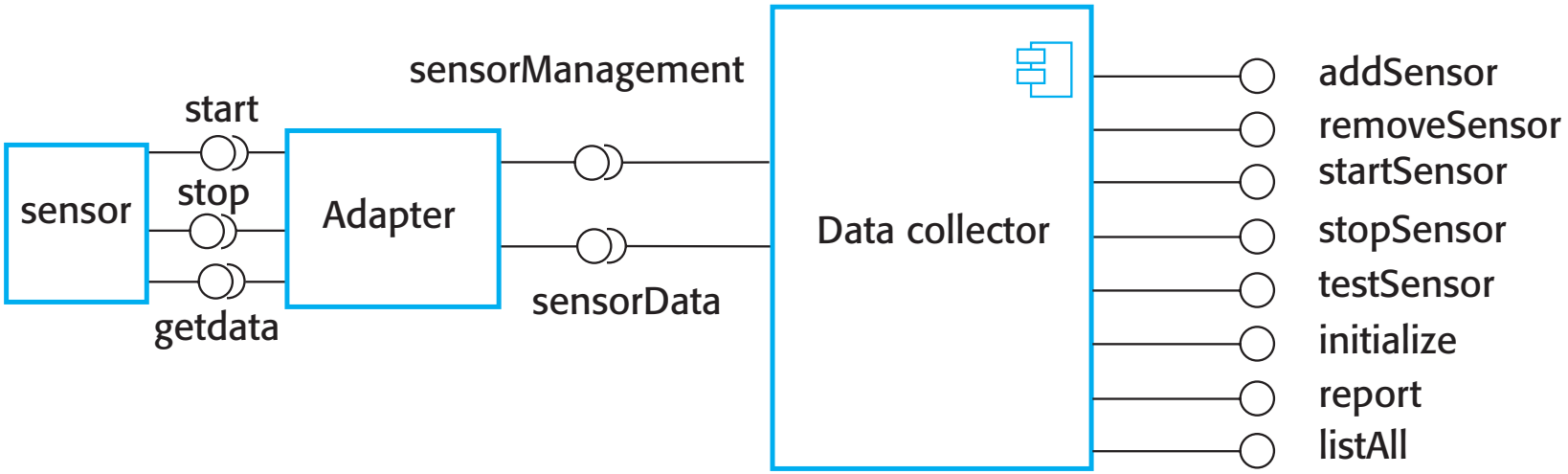
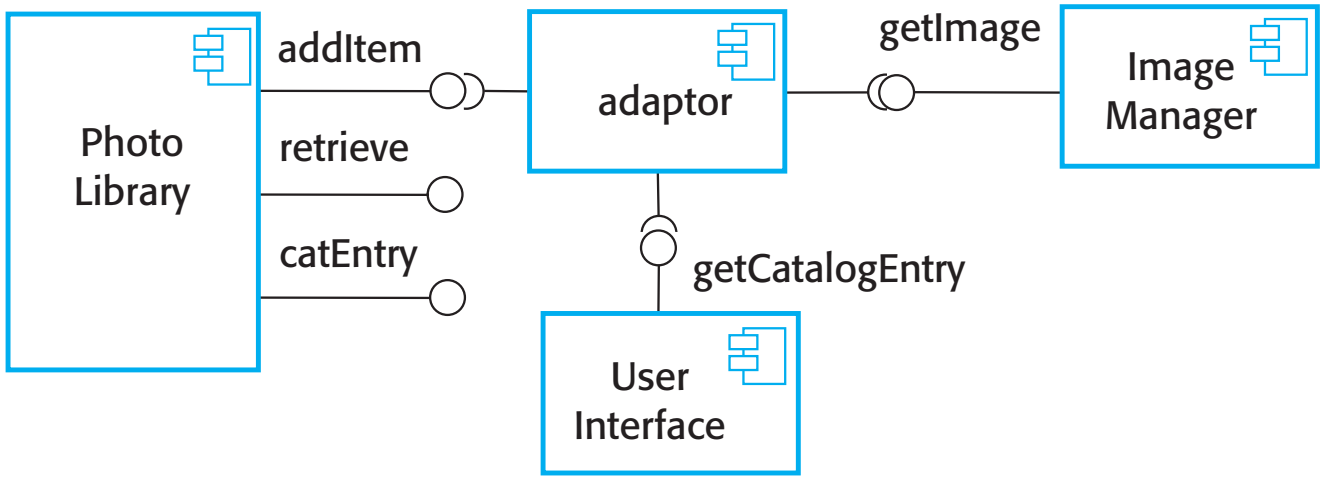


Photo library composition



Interface semantics

- You have to rely on component documentation to decide if interfaces that are syntactically compatible are actually semantically compatible.
- Consider an interface for a PhotoLibrary component:

```
public void addItem (Identifier pid ; Photograph p; CatalogEntry photodesc) ;
public Photograph retrieve (Identifier pid) ;
public CatalogEntry catEntry (Identifier pid) ;
```

Photo Library documentation

"This method adds a photograph to the library and associates the photograph identifier and catalogue descriptor with the photograph."

"what happens if the photograph identifier is already associated with a photograph in the library?"

"is the photograph descriptor associated with the catalogue entry as well as the photograph i.e. if I delete the photograph, do I also delete the catalogue information?"

The Object Constraint Language

- The Object Constraint Language (OCL) has been designed to define constraints that are associated with UML models.

- It is based around the notion of pre and post condition specification – common to many formal methods.

The OCL description of the Photo Library interface

-- The context keyword names the component to which the conditions apply

context addItem

-- The preconditions specify what must be true before execution of addItem
pre: PhotoLibrary.libSize() > 0
PhotoLibrary.retrieve(pid) = null

-- The postconditions specify what is true after execution
post:libSize () = libSize()@pre + 1
PhotoLibrary.retrieve(pid) = p
PhotoLibrary.catEntry(pid) = photodesc

context delete

pre: PhotoLibrary.retrieve(pid) <> null ;

post: PhotoLibrary.retrieve(pid) = null
PhotoLibrary.catEntry(pid) = PhotoLibrary.catEntry(pid)@pre
PhotoLibrary.libSize() = libSize()@pre—1

Photo library conditions

- As specified, the OCL associated with the Photo Library component states that:
  - There must not be a photograph in the library with the same identifier as the photograph to be entered;
  - The library must exist - assume that creating a library adds a single item to it;
  - Each new entry increases the size of the library by 1;
  - If you retrieve using the same identifier then you get back the photo that you added;
  - If you look up the catalogue using that identifier, then you get back the catalogue entry that you made.

Composition trade-offs

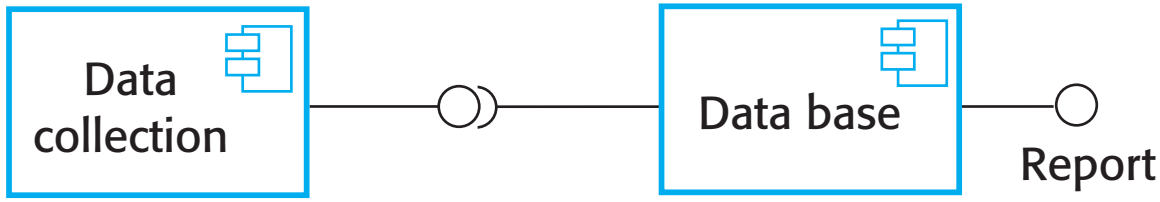
- When composing components, you may find conflicts between
  - functional VS non-functional requirements
  - the need for rapid delivery VS system evolution

- You need to make decisions such as:
  - What composition of components is effective for delivering the functional requirements while fulfilling the non-functional requirements?
  - What composition of components allows for future change?
  - What will be the emergent properties of the composed system?

Data collection and report generation components

(a)

(b)



Database component to manage data with built-in reporting facilities (e.g., Microsoft Access)

Key points

- CBSE is a reuse-based approach to defining and implementing loosely coupled components into systems.
- A component is a software unit whose functionality and dependencies are completely defined by its interfaces.
- Components may be implemented as executable elements included in a system or as external services.
- A component model defines a set of standards that component providers and composers should follow.
- The key CBSE processes are CBSE for reuse and CBSE with reuse.

Key points

- During the CBSE process, the processes of requirements engineering and system design are interleaved.
- Component composition is the process of 'wiring' components together to create a system.
- When composing reusable components, you normally have to write adaptors to reconcile different component interfaces.
- When choosing compositions, you have to consider required functionality, non-functional requirements and system evolution.

Distributed systems

Topics covered by these slides

Software reuse

- Reuse-based Software Engineering
- The reuse landscape
- Implementing software reuse

Component-based software engineering (very briefly)

- Components and component models
- CBSE processes
- Component composition

Distributed software engineering

- Distributed systems issues
- Client–server computing
- Architectural patterns for distributed systems
- Software as a service

Distributed Systems

- Virtually all large computer-based systems are now distributed systems.
  - "… a collection of independent computers that to some extent (see Transparency in next slides) appears to the user as a single coherent system."

- Information processing is distributed over several computers rather than confined to a single machine.

- Distributed Software Engineering (DSE) is therefore very important for enterprise computing systems.

SoSE ← → DSE

Another definition from wikipedia

- A distributed system is a model in which components located on networked computers communicate and coordinate their actions by passing messages.

- The components interact with each other in order to achieve a common goal.

- Three significant characteristics of distributed systems are:
  - concurrency of components,
  - lack of a global clock,
  - independent failure of components

The terms concurrent computing, parallel computing, and distributed computing have a lot of overlap, and no clear distinction exists between them.

Distributed system characteristics

- Resource sharing
  - Sharing of hardware and software resources.
- Openness
  - Use of equipment and software from different vendors.
- Concurrency
  - Concurrent processing to enhance performance.
- Scalability
  - Increased throughput by adding new resources.
- Fault tolerance
  - The ability to continue in operation after a fault has occurred.

Distributed systems issues

- Distributed systems are more complex than systems that run on a single processor.
- Complexity also arises because different parts of the system are independently managed as is the network.
- There might be no single authority in charge of the system so top-down control might be impossible.

Design issues

- Transparency
  - To what extent should the distributed system appear to the user as a single system? (next slide)
- Openness
  - Should a system be designed using standard protocols that support interoperability?
- Scalability
  - How can the system be constructed so that it is scalable?
- Security
  - How can usable security policies be defined and implemented?
- Quality of service
  - How should the quality of service be specified.
- Failure management
  - How can system failures be detected, contained and repaired?

Transparency

- Ideally, users should not be aware that a system is distributed and services should be independent of distribution characteristics.

- In practice, this is impossible because parts of the system are independently managed and because of network delays.
  - Often better to make users aware of distribution so that they can cope with problems (see our project CHOReVOLUTION)

- To achieve transparency, resources should be abstracted and addressed logically rather than physically
  - Middleware maps logical to physical resources.

http://www.chorevolution.eu

Openness

- Open distributed systems are systems that are built according to generally accepted standards.
- Components from any supplier can be integrated into the system and can inter-operate with the other system components.
- Openness implies that system components can be independently developed in any programming language and, if these conform to standards, they will work with other components.
- Web Service standards for Service-oriented Architectures (SOA) were developed to be open standards.

Scalability

- The scalability of a system reflects its ability to deliver a high quality service as demands on the system increase
  - Size -- It should be possible to add more resources to a system to cope with increasing numbers of users (load balancing).
  - Distribution -- It should be possible to geographically disperse (spatial distribution) the components of a system without degrading its performance (very difficult).
  - Manageability -- It should be possible to manage a system as it increases in size, even if parts of the system are located in independent organizations (cross-cutting responsibility, distributed governance managing, e.g., interaction and decision-making).

- There is a distinction between scaling-up and scaling-out:
  - Scaling up is more powerful system
  - Scaling out is more system instances

Security

- When a system is distributed, the number of ways that the system may be attacked is significantly increased, compared to centralized systems.

- If a part of the system is successfully attacked then the attacker may be able to use this as a 'back door' into other parts of the system.

- Difficulties in a distributed system arise because different organizations may own parts of the system (see manageability in previous slide).
  - These organizations may have mutually incompatible security policies and security mechanisms
  - See our project CHOReVOLUTION

http://www.chorevolution.eu

Types of attack

- The types of attack that a distributed system must defend itself against are:
  - Interception, where communications between parts of the system are intercepted by an attacker so that there is a loss of confidentiality.
  - Interruption, where system services are attacked and cannot be delivered as expected.
    - Denial of service attacks involve bombarding a node with illegitimate service requests so that it cannot deal with valid requests.
  - Modification, where data or services in the system are changed by an attacker.
  - Fabrication, where an attacker generates information that should not exist and then uses this to gain some privileges.

Quality of service

- The quality of service (QoS) offered by a distributed system reflects the system's ability to deliver its services dependably and with a response time and throughput that is acceptable to its users.

- Because of spatial distribution, quality of service is particularly critical when the system is dealing with time-critical data such as sound or video streams.
  - In these circumstances, if the quality of service falls below a threshold value then the sound or video may become so degraded that it is impossible to understand.

Failure management

- In a distributed system, it is inevitable that failures will occur, so the system has to be designed to be resilient to these failures.
  - "You know that you have a distributed system when the crash of a system that you've never heard of stops you getting any work done."

- Distributed systems should include mechanisms for discovering if a component of the system has failed, should continue to deliver as many services as possible in spite of that failure and, as far as possible, automatically recover from the failure.

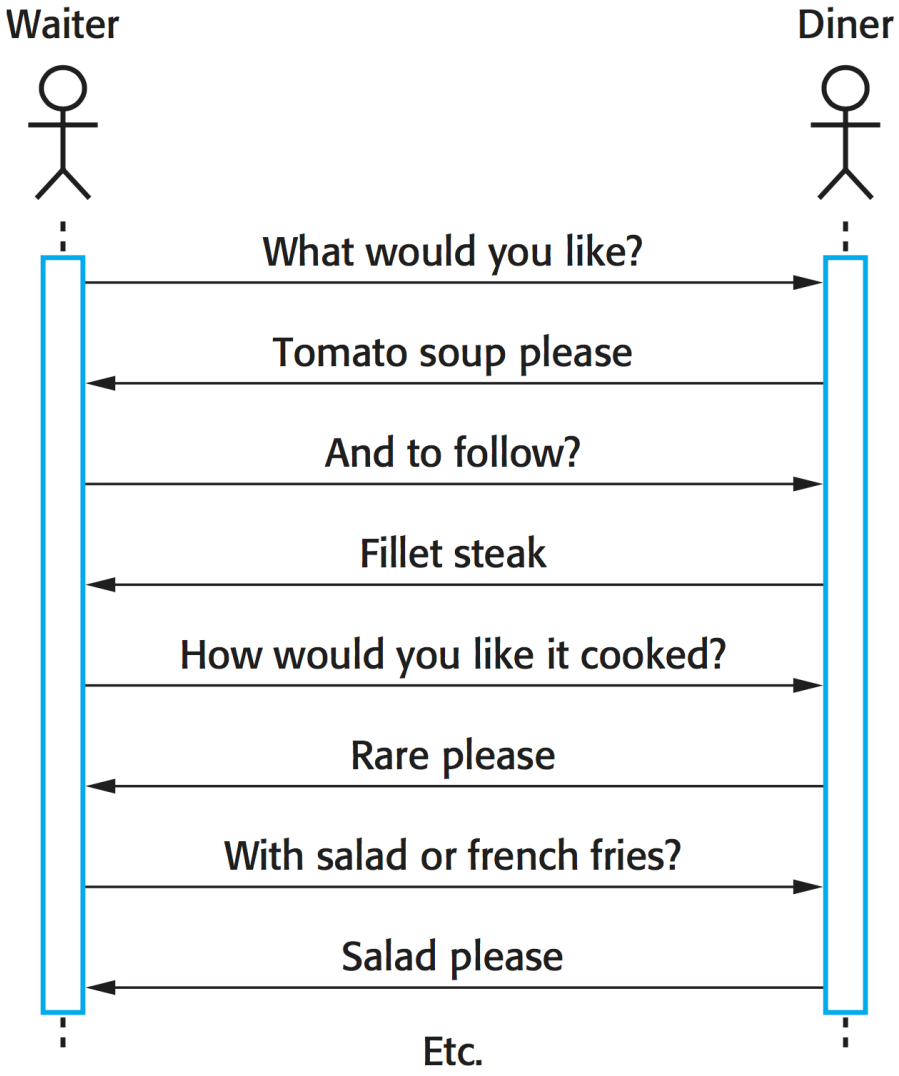
Models of interaction

At the modeling level, and from the programmer point of view, two types of interaction between "components" in a distributed system can be distinguished

- Procedural interaction, where one computer calls on a known service offered by another computer and waits for a response.
- Message-based interaction, involves the sending computer sending information about what is required to another computer. There is no necessity to wait for a response.

https://it.wikipedia.org/wiki/Remote_Method_Invocation

Procedural interaction between a diner and a waiter



Message-based interaction between a waiter and the kitchen

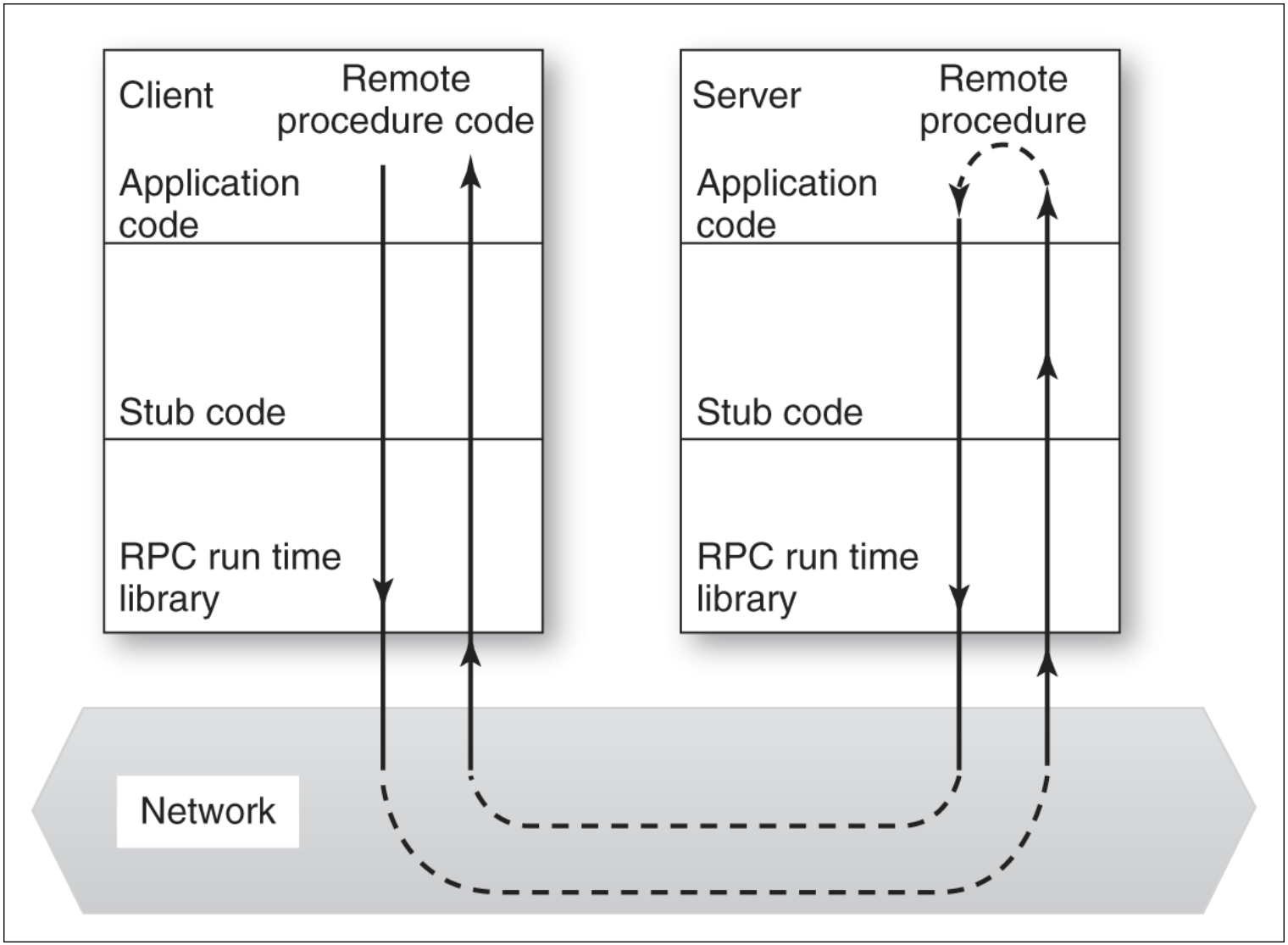
```
<starter>
   <dish name = “soup” type = “tomato” />
   <dish name = “soup” type = “fish” />
   <dish name = “pigeon salad” />
</starter>
<main course>
   <dish name = “steak” type = “sirloin” cooking = “medium” />
   <dish name = “steak” type = “fillet” cooking = “rare” />
   <dish name = “sea bass”>
</main>
<accompaniment>
   <dish name = “french fries” portions = “2” />
   <dish name = “salad” portions = “1” />
</accompaniment>
```

Remote Procedure Calls

Procedural communication in a distributed system is implemented using remote procedure calls (RPC).

- In a remote procedure call, one component calls another component as if it was a local procedure or method. The middleware in the system intercepts this call and passes it to a remote component.
- This carries out the required computation and, via the middleware, returns the result to the calling component.
- A problem with RPCs is that the caller and the callee need to be available at the time of the communication, and they must know how to refer to each other.
- RPC uses a request/wait-for-reply model.
- The RPC style leads to tight coupling of interfaces and applications
- RPC leads to point-to-point communication and as such is more appropriated for one-to-one system integration, rather than one-to-many

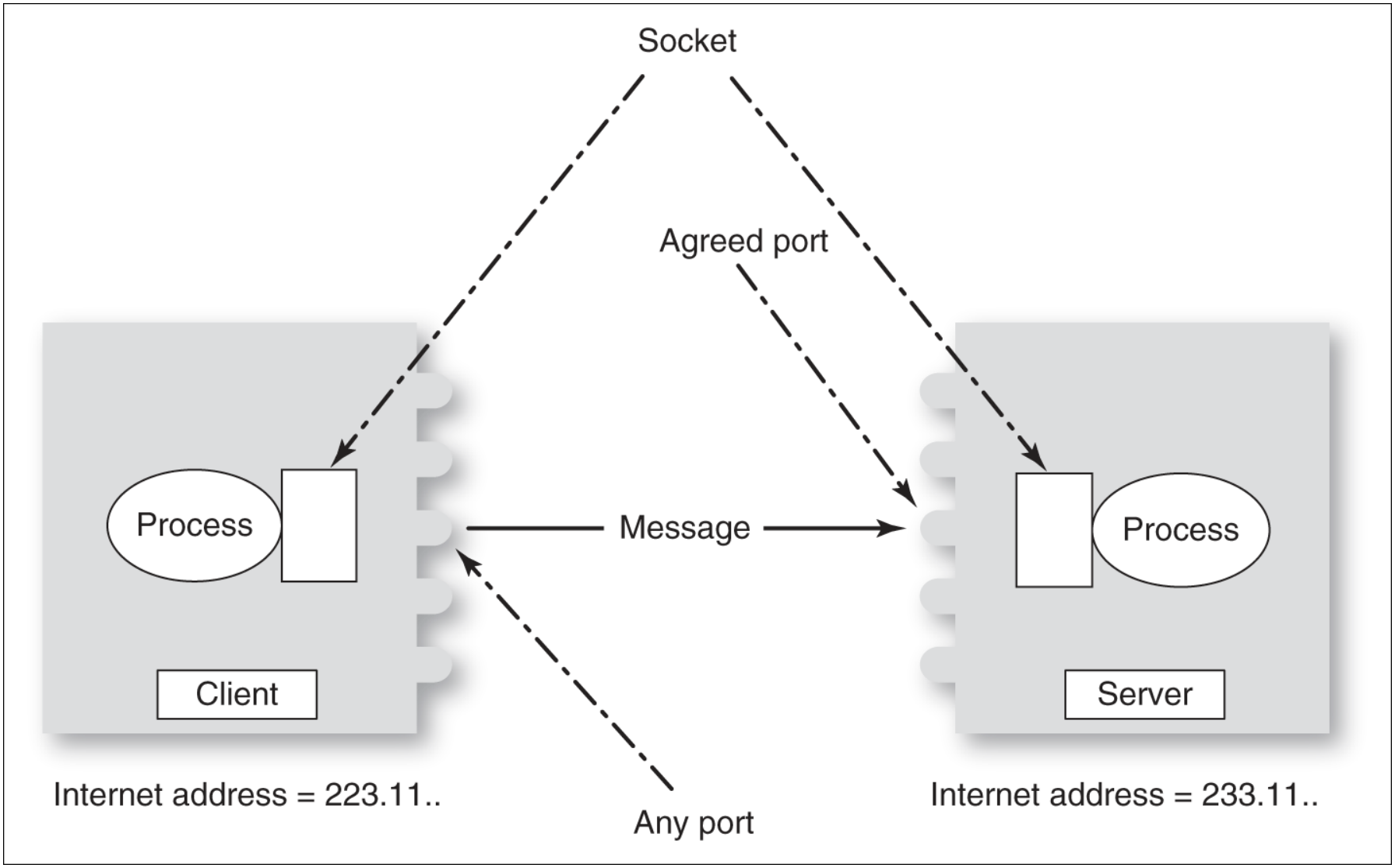
RPC communication



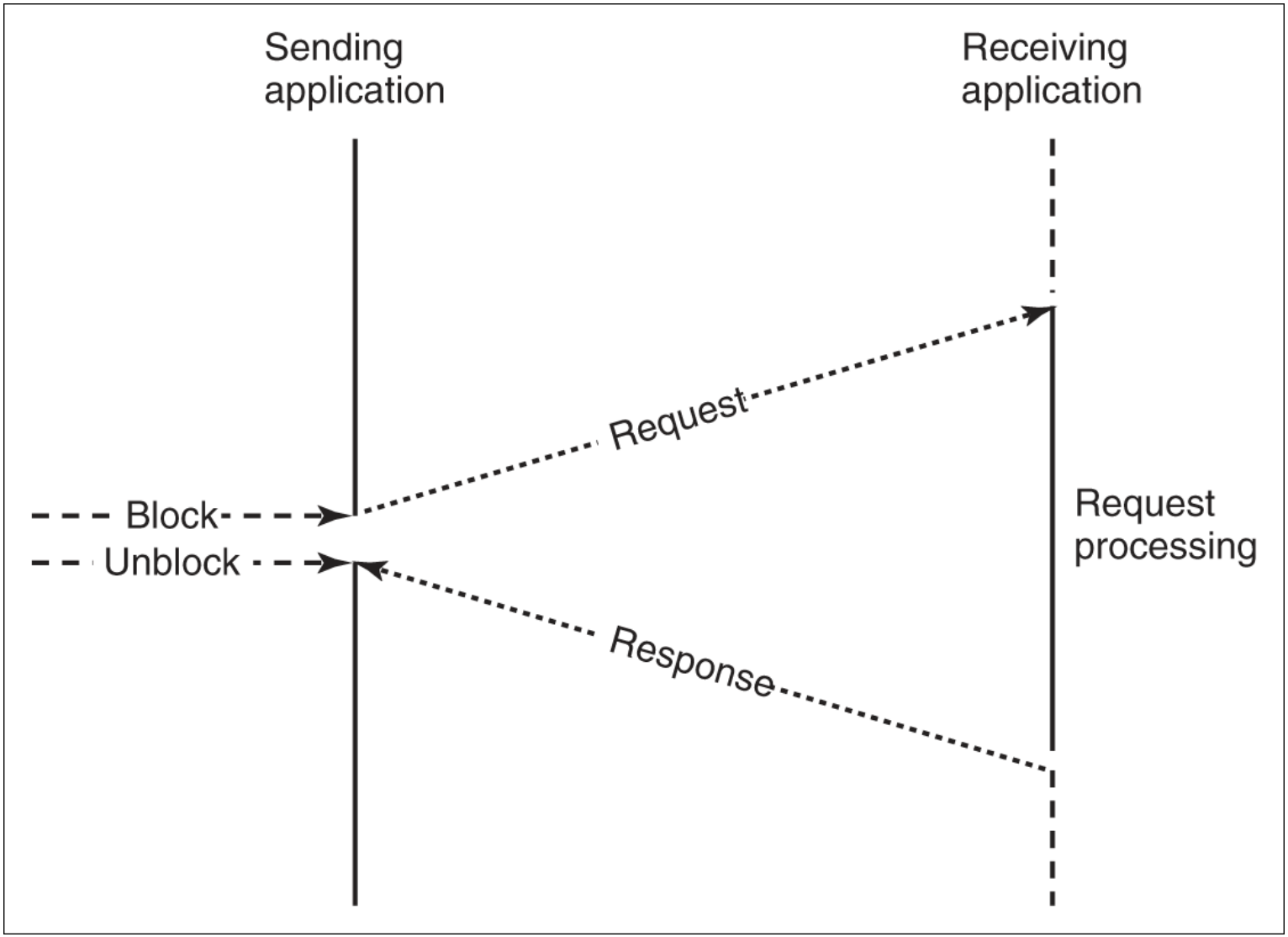
Message passing

- Message-based interaction normally involves one component creating a message that details the services required from another component.
- Through the system middleware, this is sent to the receiving component.
- The receiver parses the message, carries out the computations and creates a message for the sending component with the required results.
- In a message-based approach, it is not necessary for the sender and receiver of the message to be aware of each other. They simply communicate with the middleware.

At a lower level: Ports and sockets



Synchronous messaging

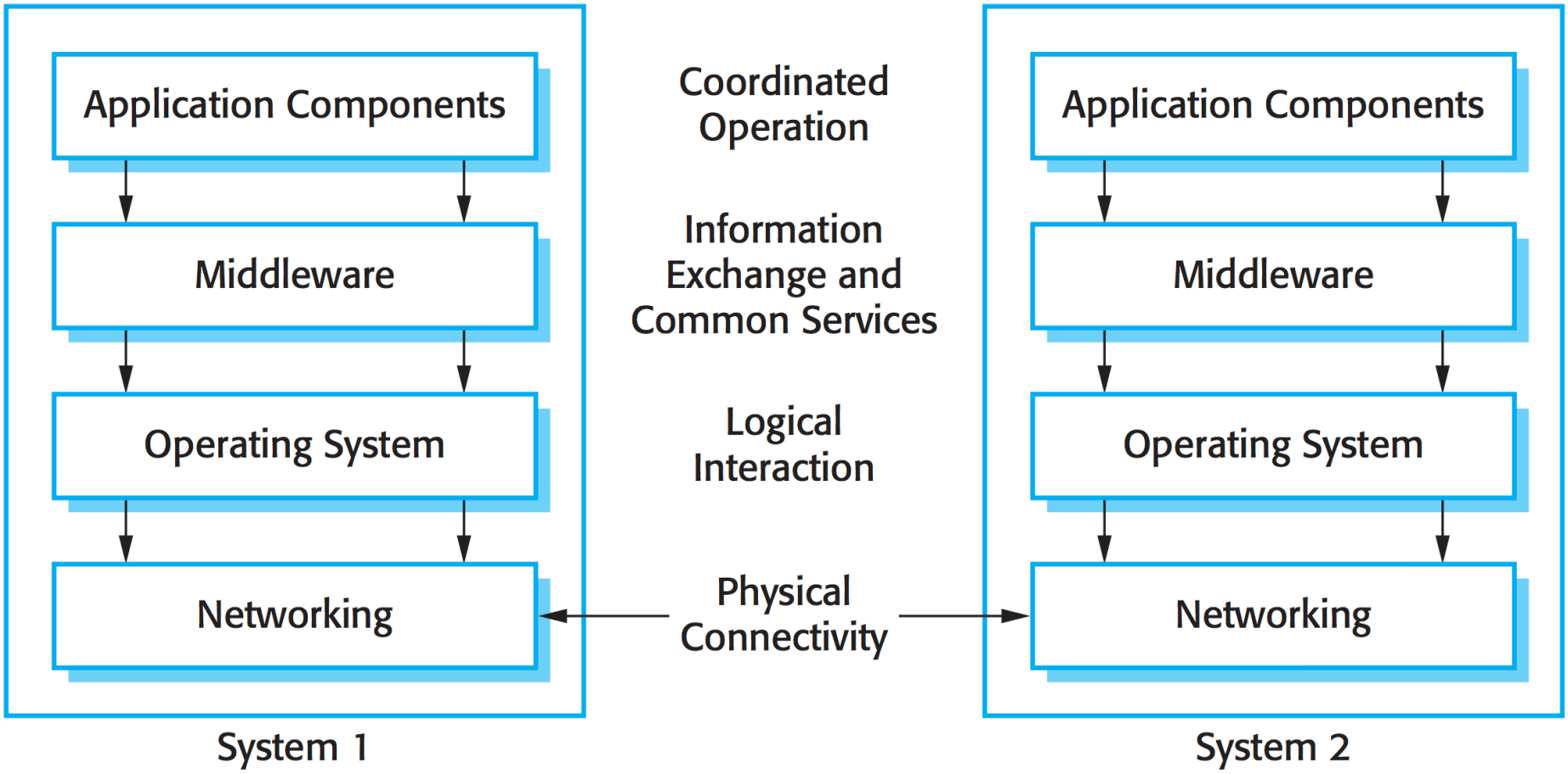


Middleware

- The components in a distributed system may be implemented in different programming languages and may execute on completely different types of processor. Models of data, information representation and protocols for communication may all be different.

- Middleware is software that can manage these diverse parts, and ensure that they can communicate and exchange data.

Middleware in a distributed system



Middleware support

- Interaction support, where the middleware coordinates interactions between different components in the system
  - The middleware provides location transparency in that it isn't necessary for components to know the physical locations of other components.

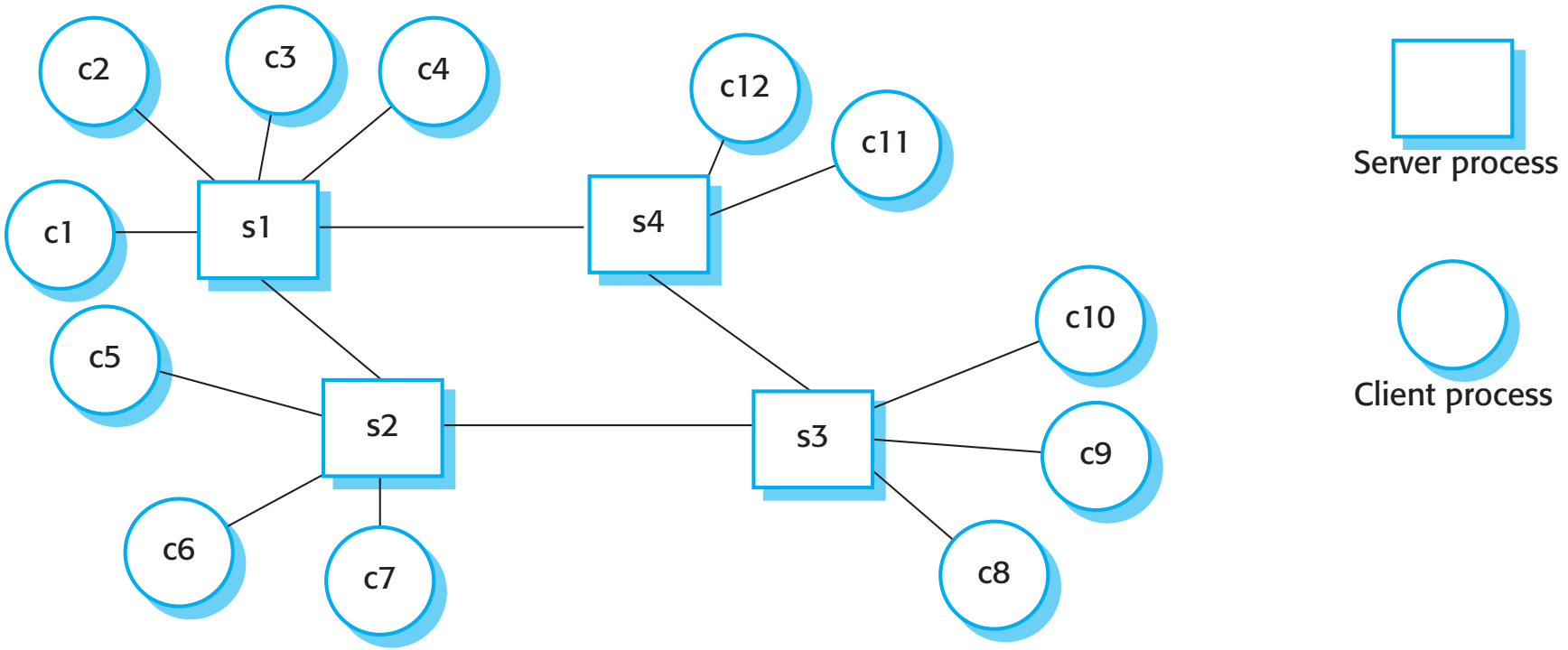
- The provision of common services, where the middleware provides (reusable implementations of) services that may be required by several components in the distributed system.
  - By using these common services, components can easily inter-operate and provide user services in a consistent way.

Client-server computing

- Distributed systems that are accessed over the Internet are normally organized as client-server systems.
- In a client-server system, the user interacts with a program running on their local computer (e.g. a web browser or mobile application). This interacts with another program running on a remote computer (e.g. a web server).
- The remote computer provides services, such as access to web pages, which are available to external clients.

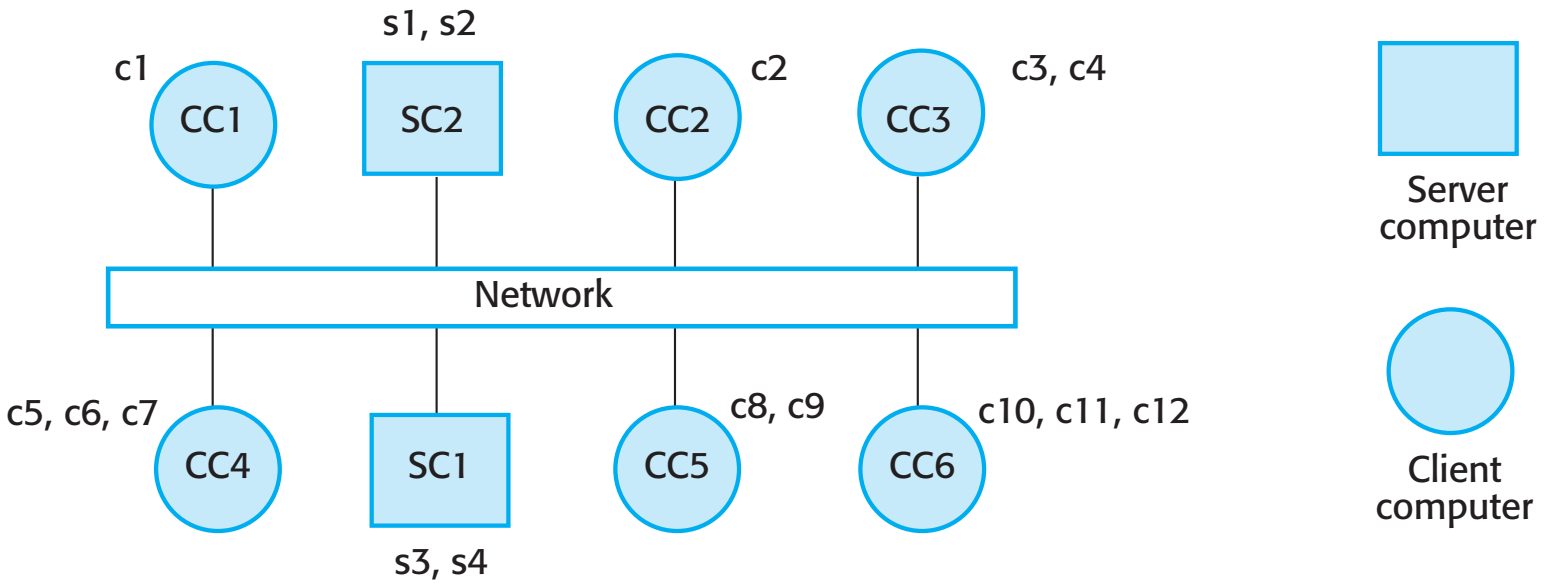
Client–server interaction

- Logical process view



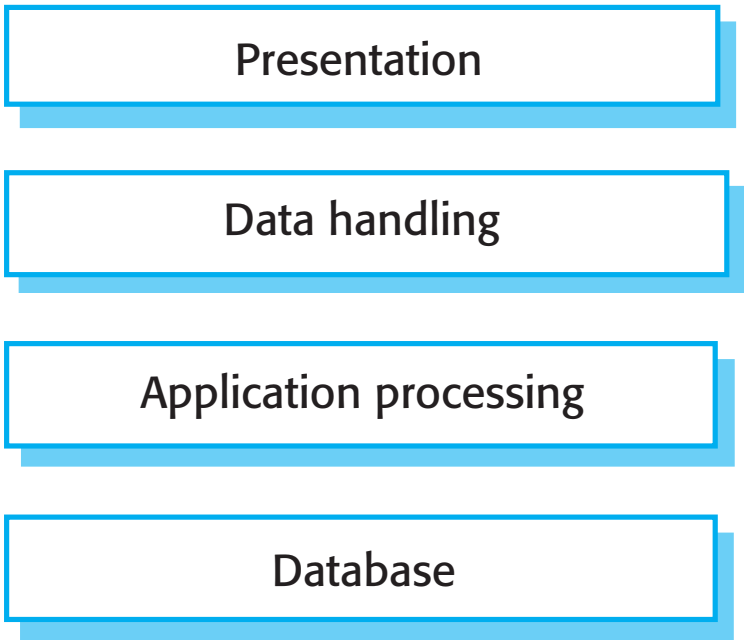
Mapping of clients and servers to networked computers

- Physical view: concrete mapping to spatially distributed physical server machine



Layered architectural model for client–server applications

- Presentation
  - concerned with presenting information to the user and managing all user interaction.
- Data handling
  - manages the data that is passed to and from the client. Implement checks on the data, generate web pages, etc.
- Application processing layer
  - concerned with implementing the logic of the application and so providing the required functionality to end users.
- Database
  - Stores data and provides transaction management services, etc.



Architectural patterns for distributed systems

- Widely used ways of organizing the architecture of a distributed system:
  - Master-slave architecture, which is used in real-time systems in which guaranteed interaction response times are required.
  - Two-tier client-server architecture, which is used for simple client-server systems, and where the system is centralized for security reasons.
  - Multi-tier client-server architecture, which is used when there is a high volume of transactions to be processed by the server.
  - Distributed component architecture, which is used when resources from different systems and databases need to be combined, or as an implementation model for multi-tier client-server systems.
  - Peer-to-peer architecture, which is used when clients exchange locally stored information and the role of the server is to introduce clients to each other

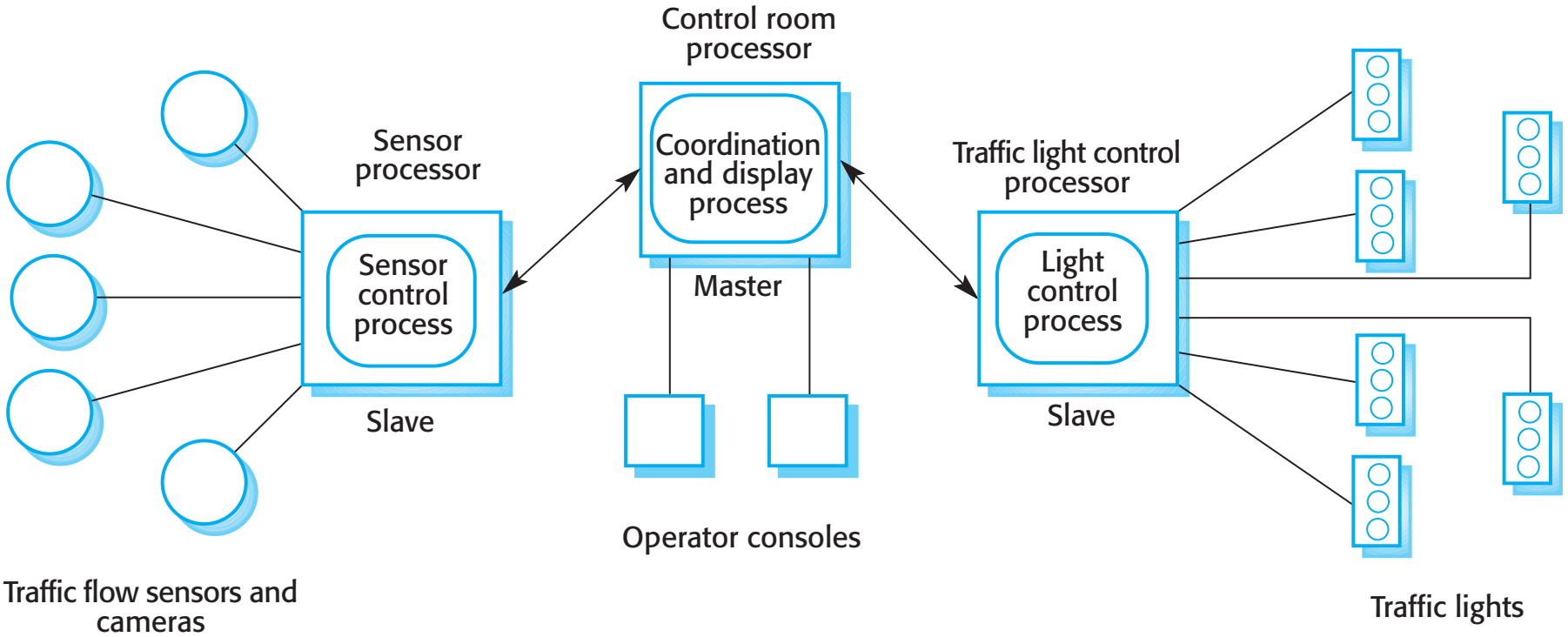
Master-slave architectures

- Master-slave architectures are commonly used in real-time systems where there may be separate processors associated with data acquisition from the system's environment, data processing and computation and actuator management.

- The 'master' process is usually responsible for computation, coordination and communications and it controls the 'slave' processes.

- 'Slave' processes are dedicated to specific actions, such as the acquisition of data from an array of sensors.

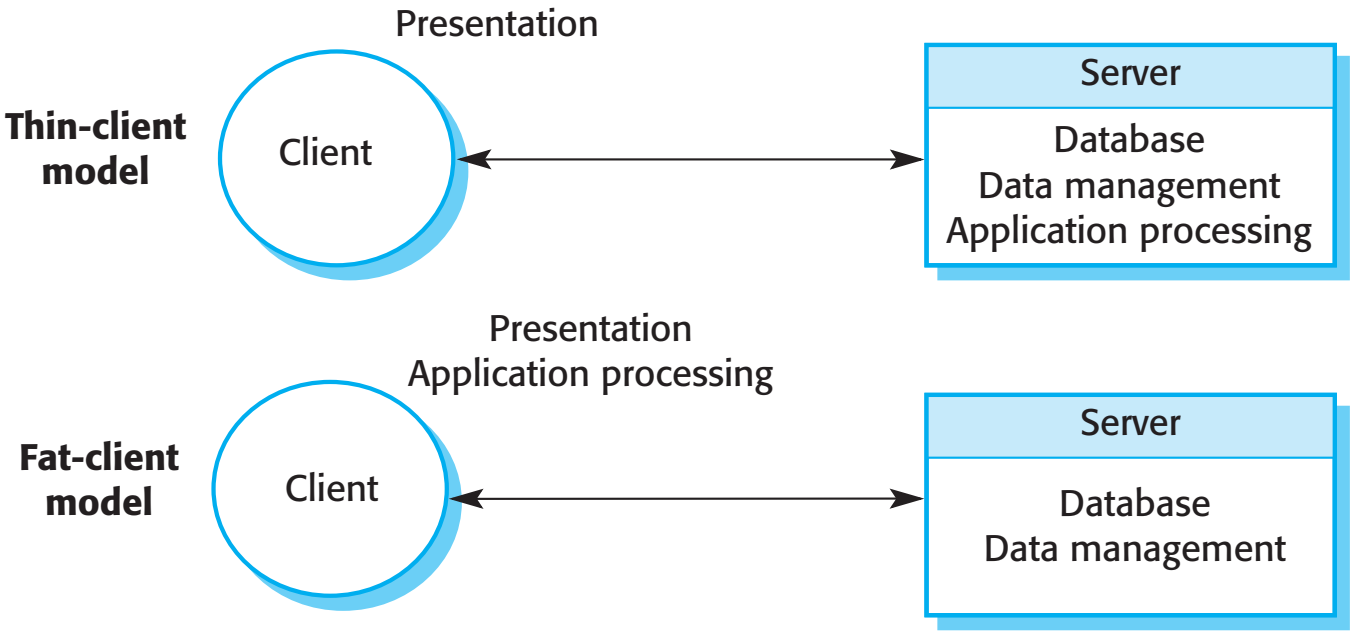
A traffic management system with a master-slave architecture



Two-tier client server architectures

- In a two-tier client-server architecture, the system is implemented as a single logical server plus an indefinite number of clients that use that server.
  - Thin-client model, where the presentation layer is implemented on the client and all other layers (data management, application processing and database) are implemented on a server.
  - Fat-client model, where some or all of the application processing is carried out on the client. Data management and database functions are implemented on the server.

Thin- and fat-client architectural models



Thin client model

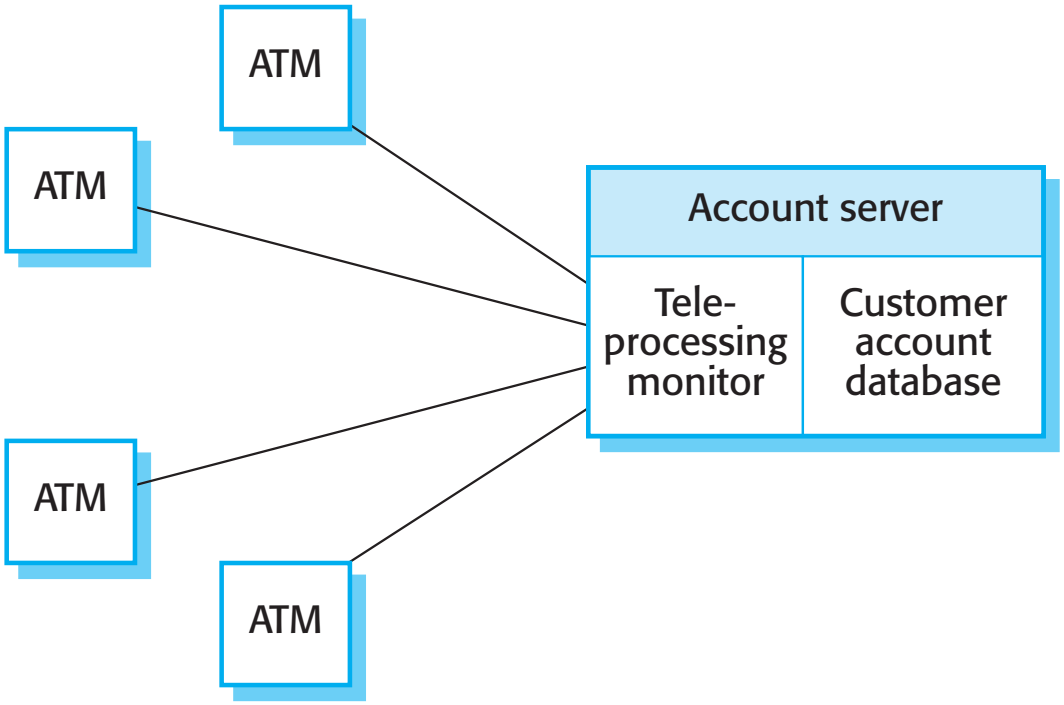
- A major disadvantage is that it places a heavy processing load on both the server and the network.

- Used when legacy systems are migrated to client server architectures.
  - The legacy system acts as a server in its own right with a graphical interface implemented on a client.

Fat client model

- More processing is delegated to the client as the application processing is locally executed.
- Most suitable for new C/S systems where the capabilities of the client system are known in advance.
- More complex than a thin client model especially for management. New versions of the application have to be installed on all clients.

A fat-client architecture for an ATM system



Thin and fat clients

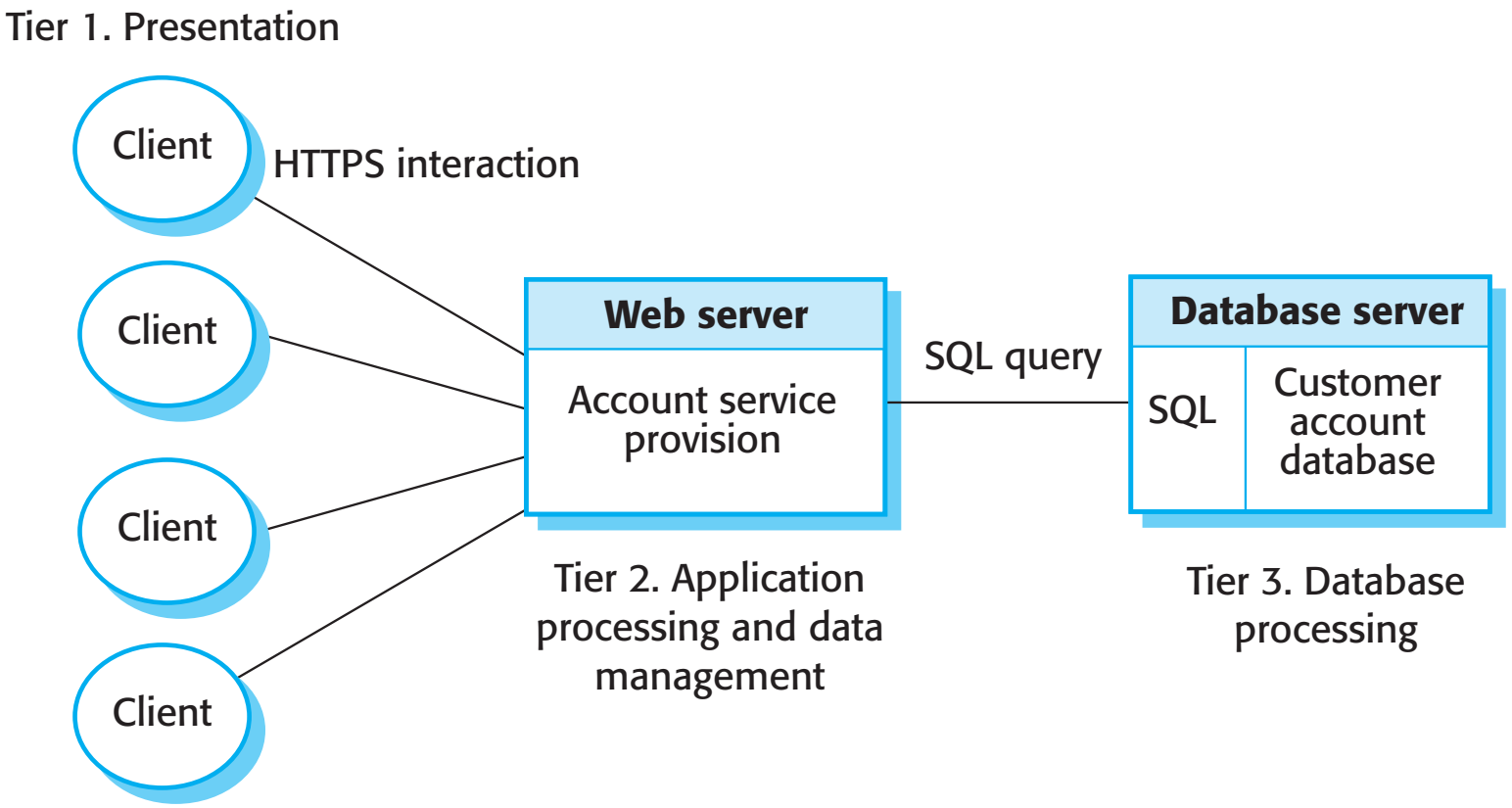
- Distinction between thin and fat client architectures has become blurred
- Javascript allows local processing in a browser so 'fat-client' functionality available without software installation
- Mobile apps carry out some local processing to minimize demands on network
- Auto-update of apps reduces management problems
- There are now very few thin-client applications with all processing carried out on remote server.

Multi-tier client-server architectures

- In a ‘multi-tier client–server’ architecture, the different layers of the system, namely presentation, data management, application processing, and database, are separate processes that may execute on different processors.

- This avoids problems with scalability and performance if a thin-client two-tier model is chosen, or problems of system management if a fat-client model is used.

Three-tier architecture for an Internet banking system



Use of client–server architectural patterns

Architecture	Applications
Two-tier client–server architecture with thin clients	Legacy system applications that are used when separating application processing and data management is impractical. Clients may access these as services, as discussed in Section 18.4. Computationally intensive applications such as compilers with little or no data management. Data-intensive applications (browsing and querying) with nonintensive application processing. Browsing the Web is the most common example of a situation where this architecture is used.

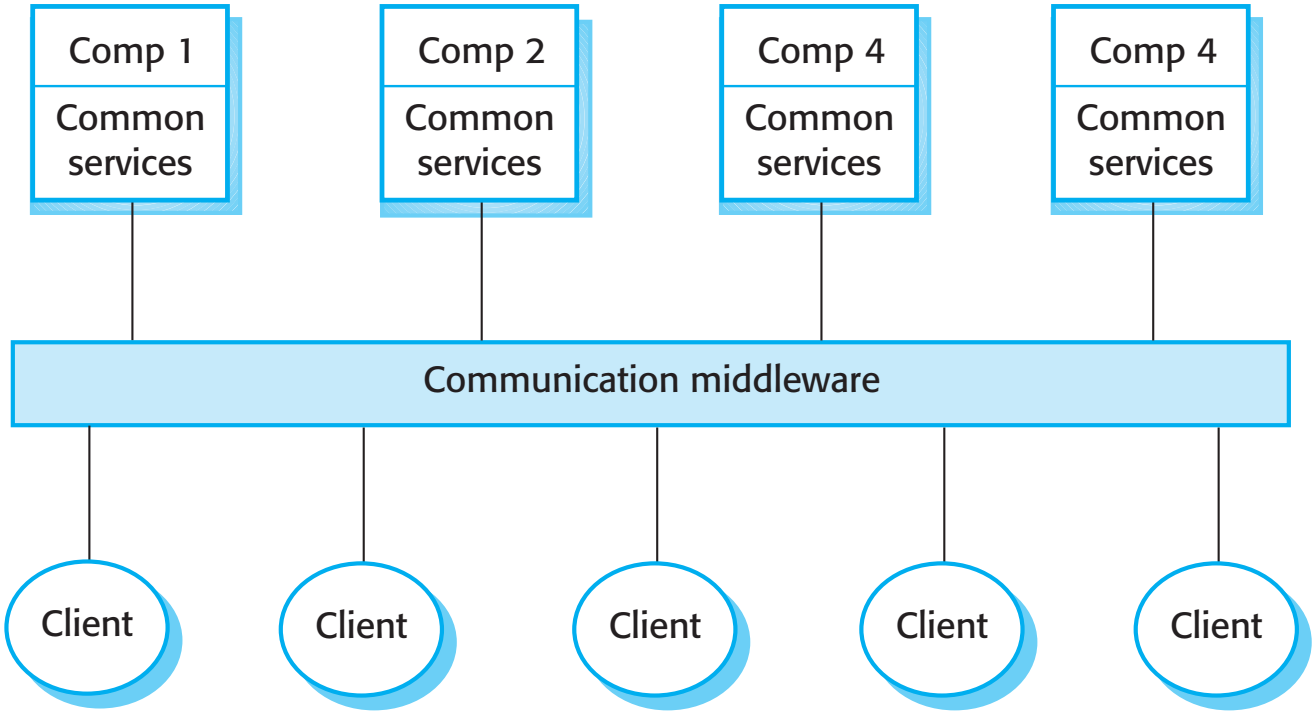
Use of client–server architectural patterns

Architecture	Applications
Two-tier client-server architecture with fat clients	Applications where application processing is provided by off-the-shelf software (e.g., Microsoft Excel) on the client. Applications where computationally intensive processing of data (e.g., data visualization) is required. Mobile applications where internet connectivity cannot be guaranteed. Some local processing using cached information from the database is therefore possible.
Multi-tier client–server architecture	Large-scale applications with hundreds or thousands of clients. Applications where both the data and the application are volatile. Applications where data from multiple sources are integrated.

Distributed component architectures

- There is no distinction in a distributed component architecture between clients and servers.
- Each distributable entity is a component that provides services to other components and receives services from other components.
- Component communication is through a middleware system.

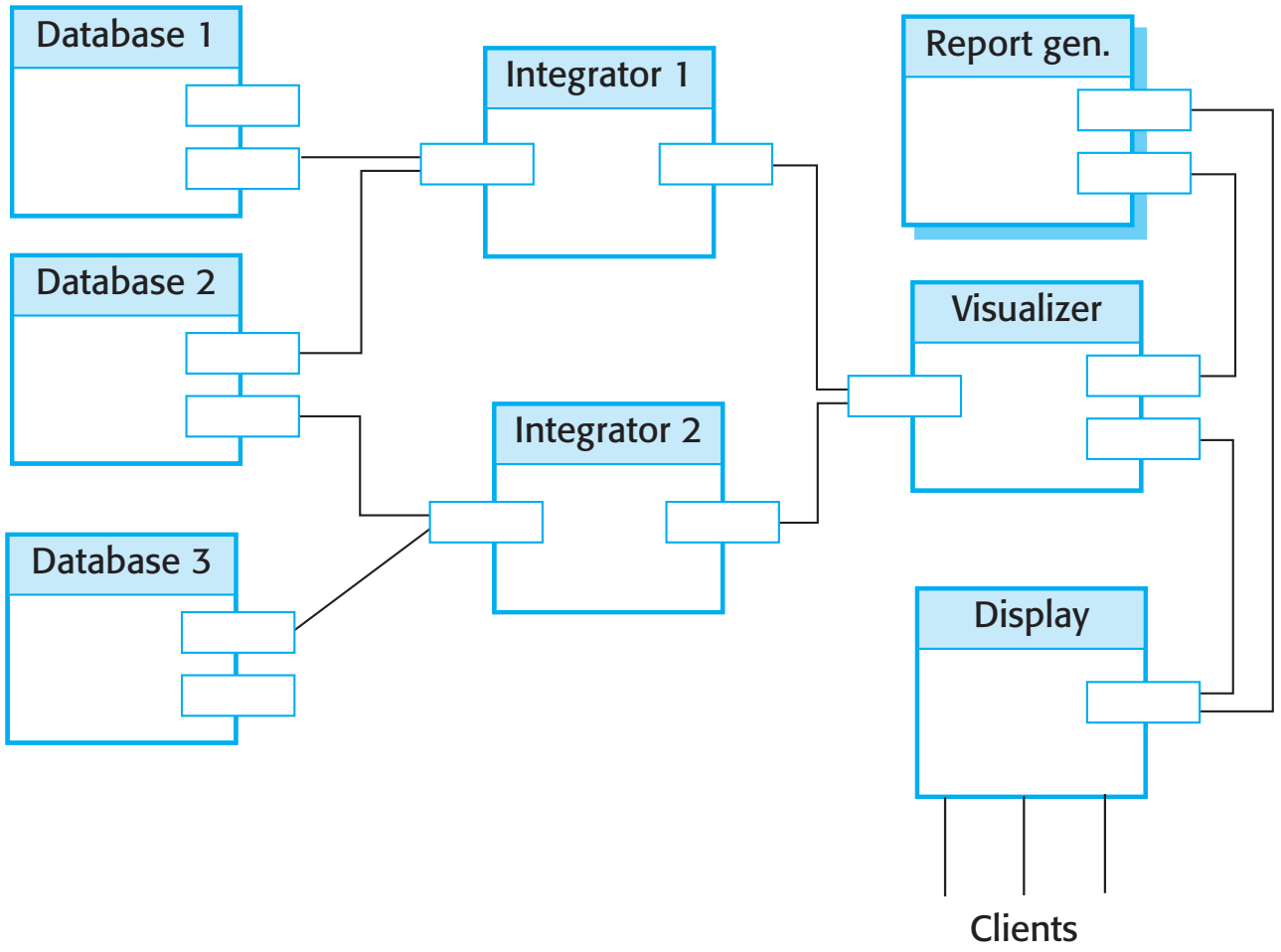
A distributed component architecture



Benefits of distributed component architecture

- It allows the system designer to delay decisions on where and how services should be provided.
- It is a very open system architecture that allows new resources to be added as required.
- The system is flexible and scalable.
- It is possible to reconfigure the system dynamically with objects migrating across the network as required.

A distributed component architecture for a data mining system



Disadvantages of distributed component architecture

- Distributed component architectures suffer from two major disadvantages:
  - They are more complex to design than client–server systems. Distributed component architectures are difficult for people to visualize and understand.

Standardized middleware for distributed component systems has never been accepted by the community. Different vendors, such as Microsoft and Sun, have developed different, incompatible middleware → YET ANOTHER INTEROPERABILITY BARRIER

As a result of these problems, service-oriented architectures have been replacing distributed component-based architectures in many situations.

Peer-to-peer architectures

- Peer to peer (p2p) systems are decentralized systems where computations may be carried out by any node in the network.

- The overall system is designed to take advantage of the computational power and storage of a large number of networked computers.

- Most p2p systems have been personal systems but there is increasing business use of this technology.

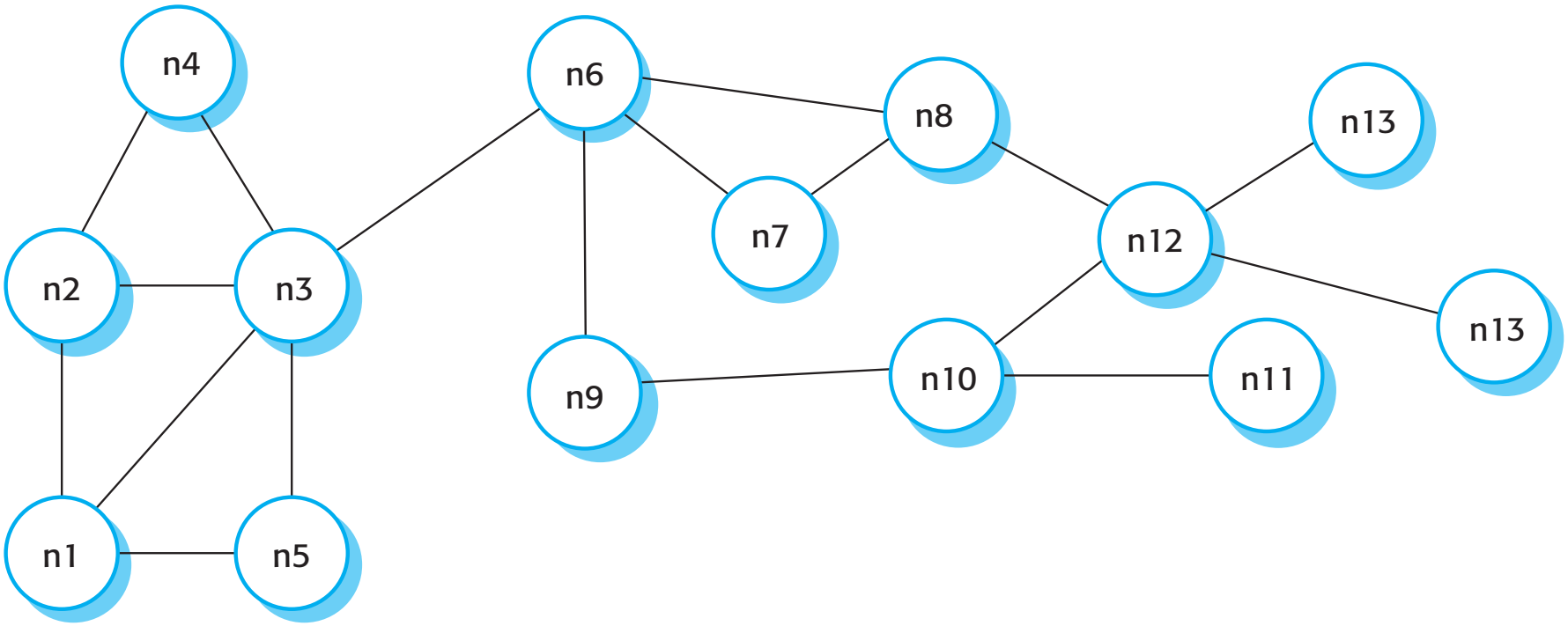
Peer-to-peer systems

- File sharing systems based on the BitTorrent protocol
- Messaging systems such as Jabber
- Payments systems – Bitcoin
- Databases – Freenet is a decentralized database
- Phone systems – Viber
- Computation systems - SETI@home

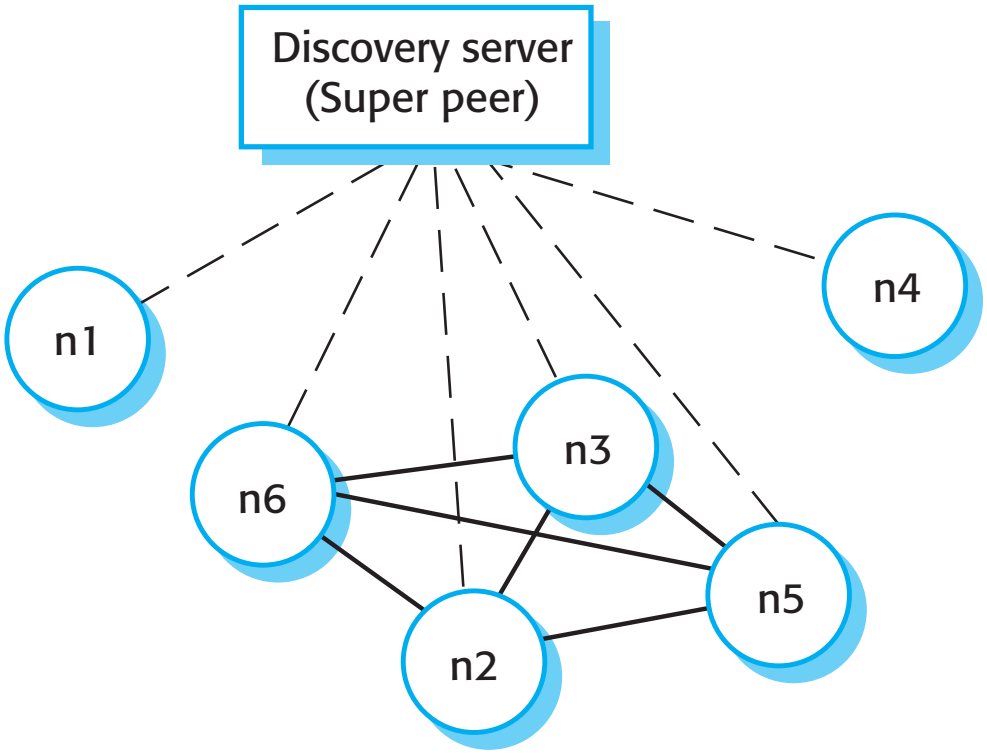
P2p architectural models

- The logical network architecture
  - Decentralised architectures;
  - Semi-centralised architectures.
- Application architecture
  - The generic organisation of components making up a p2p application.
- Focus here on network architectures.

A decentralized p2p architecture



A semicentralized p2p architecture



Use of p2p architecture

- When a system is computationally-intensive and it is possible to separate the processing required into a large number of independent computations.

- When a system primarily involves the exchange of information between individual computers on a network and there is no need for this information to be centrally-stored or managed.

Security issues in p2p system

- Security concerns are the principal reason why p2p architectures are not widely used.
- The lack of central management means that malicious nodes can be set up to deliver spam and malware to other nodes in the network.
- P2P communications require careful setup to protect local information and if not done correctly, then this is exposed to othe peers.

Software as a service

Software as a Service (SaaS) involves hosting the software remotely and providing access to it over the Internet.

- Software is deployed on a server (or more commonly a number of servers) and is accessed through a web browser (a dedicated web application). It is not deployed on a local PC.
- The software is owned and managed by a software provider, rather than the organizations using the software.
- Users may pay for the software according to the amount of use they make of it or through an annual or monthly subscription.

SaaS and SOA

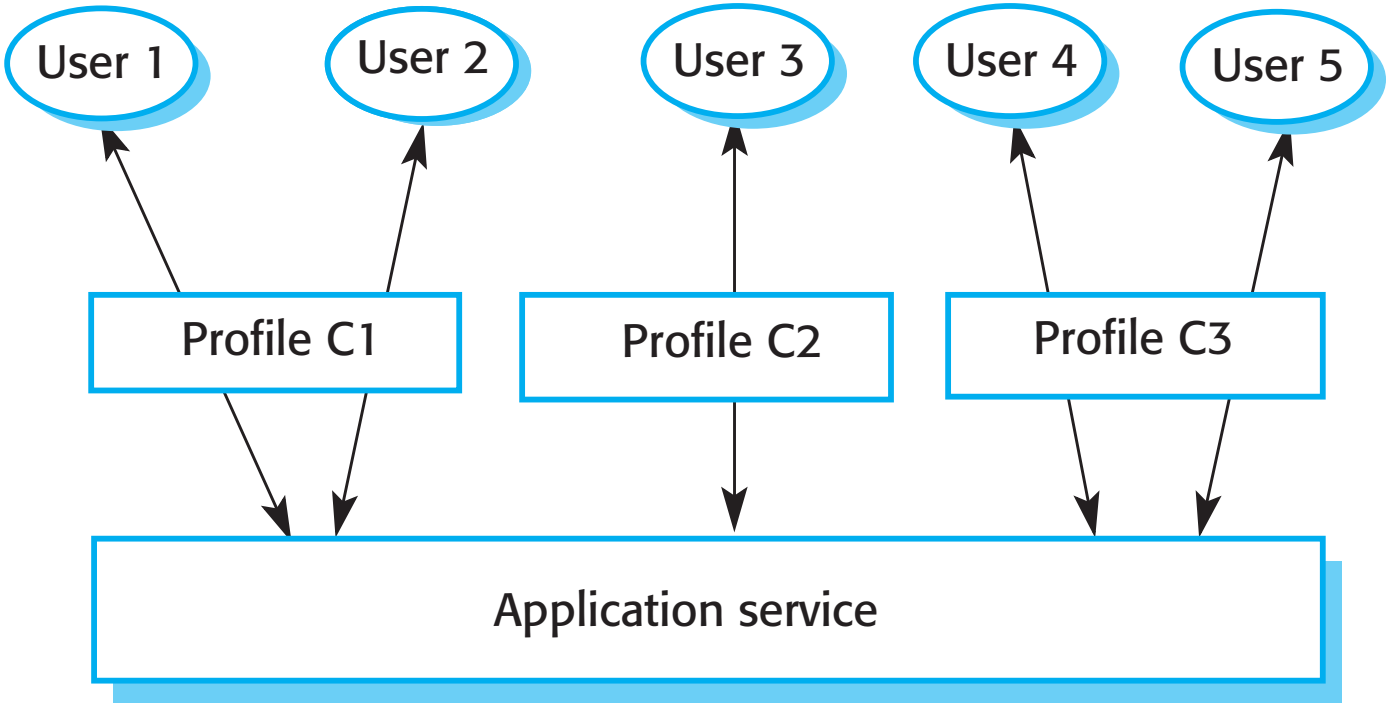
- Software as a service is a way of providing functionality on a remote server with client access through a dedicated web application. The server maintains the user's data and state during an interaction session. Transactions are usually long transactions, e.g., editing a document.

- Service-oriented architecture is an approach to structuring a software system as a set of separate, stateless services. These may be provided by multiple providers and may be distributed. Typically, transactions are short transactions where a service is called, does something then returns a result.

Implementation factors for SaaS

- Configurability
  - How do you configure the software for the specific requirements of each organization?
- Multi-tenancy
  - How do you present each user of the software with the impression that they are working with their own copy of the system while, at the same time, making efficient use of system resources?
- Scalability
  - How do you design the system so that it can be scaled to accommodate an unpredictably large number of users?

Configuration of a software system offered as a service



Service configuration

- Branding, where users from each organization, are presented with an interface that reflects their own organization.
- Business rules and workflows, where each organization defines its own rules that govern the use of the service and its data.
- Database extensions, where each organization defines how the generic service data model is extended to meet its specific needs.
- Access control, where service customers create individual accounts for their staff and define the resources and functions that are accessible to each of their users.

Multi-tenancy

- Multi-tenancy is a situation in which many different users access the same system instance and the system architecture is defined to allow the efficient sharing of system resources.
- It must appear to each user that they have the sole use of the system.
- Multi-tenancy involves designing the system so that there is an absolute separation between the system functionality and the system data.
- Multitenancy contrasts with multi-instance architectures, where separate software instances operate on behalf of different tenants

A multitenant database

Tenant	Key	Name	Address
234	C100	XYZ Corp	43, Anystreet, Sometown
234	C110	BigCorp	2, Main St, Motown
435	X234	J. Bowie	56, Mill St, Starville
592	PP37	R. Burns	Alloway, Ayrshire

Scalability

- Develop applications where each component is implemented as a simple stateless service that may be run on any server.
- Design the system using asynchronous interaction so that the application does not have to wait for the result of an interaction (such as a read request).
- Manage resources, such as network and database connections, as a pool so that no single server is likely to run out of resources.
- Design your database to allow fine-grain locking. That is, do not lock out whole records in the database when only part of a record is in use.

Key points

- The benefits of distributed systems are that they can be scaled to cope with increasing demand, can continue to provide user services if parts of the system fail, and they enable resources to be shared.

- Issues to be considered in the design of distributed systems include transparency, openness, scalability, security, quality of service and failure management.

- Client–server systems are structured into layers, with the presentation layer implemented on a client computer. Servers provide data management, application and database services.

- Client-server systems may have several tiers, with different layers of the system distributed to different computers.

Key points

- Architectural patterns for distributed systems include master-slave architectures, two-tier and multi-tier client-server architectures, distributed component architectures and peer-to-peer architectures.

- Distributed component systems require middleware to handle component communications and to allow components to be added to and removed from the system.

- Peer-to-peer architectures are decentralized with no distinguished clients and servers. Computations can be distributed over many systems in different organizations.

- Software as a service is a way of deploying applications as thin client-server systems, where the client is a web browser.